

Fast parallel index-based GPU particle collision detection for science and engineering.

Jackie Bell^{1,*}

¹University of Florida, Department of Mechanical and Aerospace engineering, Gainesville, Florida, 32609, United States of America

*jb2ndr@ufl.edu

ABSTRACT

A fast particle collision detection method by rounding and indexing particle and cell locations. The two part process is performed in compute and render pipelines exclusively on the GPU. The method is GPU-Driven in that detection and rendering are simultaneous and all logic resides on the GPU. The application can process 1 million particles in less than 1/30th of a second, (30 frames per second) on an inexpensive laptop workstation. This speed provides a basis for scientists and engineers accustomed to computational fluid dynamics, to consider particle systems for thermo-fluid flow including rarefied gases. It presents an opportunity for students and educators to explore massive particle systems (greater than 1 million particles). Also, in order to encourage comparative analysis, we have developed a light-weight benchmarking method where test data is generated by software.

1 introduction

Several trends in scientific computing are aimed at exploiting the graphics processing unit (GPU) for purposes other than computer graphics¹⁻⁴. To enable researchers from fields outside computer science, languages like CUDA were introduced. These provided an abstraction more amenable to the general computing with which scientists are accustomed⁵. It is this utilization, as a highly parallel math co-processor, that has provided the expansion in general purpose GPU computing (GPGPU)⁶.

But, the GPU originated as, and continues to be, an optimal graphics rendering device. Left to perform this connate purpose, without interference, it will launch as many tasks as possible, as efficiently as possible.

The GPU is also a separate computing platform complete with instruction set, operating system, memory, and programming language. Treating it as an adjunct to the CPU limits its potential as a fast, independent, parallel processing device. Fully employing the GPU then requires that all calculations are parallel and that there are no memory exchanges with, or calculations by, the CPU. This has developed into a movement towards *GPU-Driven*⁷ implementations.

One important utilization of GPU parallel processing is in the field of thermo-fluid analysis. Currently, this space is dominated by computational fluid dynamics (CFD), but there has been substantial growth in nodal type particle methods⁸, such as smoothed particle hydrodynamics (SPH)⁹ and the discrete element method (DEM)¹⁰.

Both of these approaches require collision detection (CD) which is often a two phase process. First, a proximity query is required to determine the number of nearest neighbors in a region of interest (RI) and from this information a potentially colliding set (PCS) is constructed. This *k-nearest neighbor* search (KNN) is computationally expensive as it exhibits quadratic complexity - aggravated by the fact that simulation accuracy requires massive numbers of particles. In the next phase the PCS is evaluated to determine actual contact.

There have been a number of attempts to accelerate CD on the GPU, which according to some authors consumes 50-90% of computing resources¹¹. This computational expense makes CD the main impediment to massive particle systems. Currently, very large sets of particles, such as a billion or more, require high performance computing - and even then are time consuming.

Collision detection is a wide subject spanning applications like *particle systems* (PS) in *computer graphics animation* (CGA) for entertainment, computer aided design (CAD), thermo-fluid engineering, molecular dynamics (MD), and N-Body systems. This diversity defies common bench-marking but in the sub-set of particle-particle collisions the commonality allows a bench-marking method valid regardless of CPU-GPU combinations. We therefore propose a light weight bench-marking regime based on the *minimum machine rendering rate (mmrr)*.

Ultimately, we believe, particle collision detection is about decreasing the cost of hardware required for simulating and studying increasing numbers of particles, in turn increasing the availability of particle systems to researchers and students. There is certainly demand as the growth of papers, and the establishment of a journal¹¹, on the subject of nodal particle methods indicates (fig. 1). But in a review of particle collision detection on the GPU only 95 papers were found between 1989 and 2023

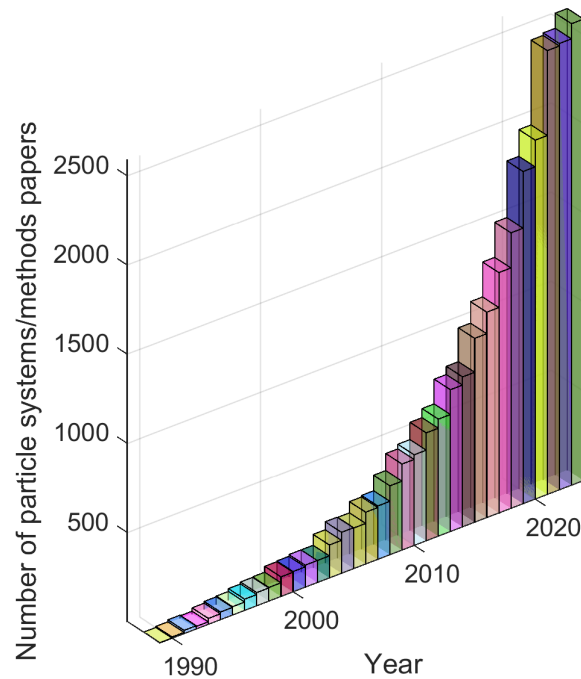


Figure 1. Number of publications related to particle simulation in the recent 30 years, obtained from the Web of Science with the following keywords: "discrete element method", "material point method" and "smoothed particle". Compared to 95 papers on particle collision detection in the same period¹². Reproduced in part from Zue et.al.⁸.

concerning accelerating *exact* particle collision detection specifically¹²; an astonishing dearth of supply relative to demand. A common language and process would also facilitate communication and comparative analysis providing that synergistic effect when so many people can apply themselves to a single problem.

An important audience for this approach are those involved with nodal type particle systems and CFD experts who may not have much computer graphics experience. Some effort is then devoted to basic GPU architecture and Vulkan programming with the goal of encouraging wider participation with this powerful approach.

2 Background

Over the years, since the inception of the GPU, a number of authors have attempted to perform collision detection in *image space*¹³. Image space is the projection of a 3-dimensional space onto two dimensions. In the process of 'flattening' a virtual scene, a number of 3-dimensional structures are utilized. Image space methods attempt to re-purpose these *render attachments* to act as computational spaces. There have been efforts to use the depth-buffer or z-buffer⁴, or texture memory¹⁴ to store variables, or perform calculations, alongside graphics processing. Image space techniques are born of the urge to 'piggyback' onto the GPU image processing scheme to perform mathematics more effectively. There are a number of reports on applications that are executed on the GPU exclusively^{10, 15–18}. Some image space techniques adapt GPU rendering mechanics, such as ray-tracing, to perform approximate CD¹⁹.

Aspiring image space solutions focus heavily on collision detection^{1, 2, 4, 17, 20}, where collision detection is defined as *the determination of which objects, in a set of unordered objects, have intersecting regions of interest, and if these regions intersect, how and when will the intersecting objects be in contact*. Regions of interest (RI)^{21, 22} can take any form in 2-d, 3-d, or 4-d space-time,^{23–28}. Determining when one object is in the region of interest (RI) of any other, in a set of objects, is a computationally expensive *k*-near neighbors (KNN) search. The naive pair-wise, or brute-force, search has complexity of $O(n^2)$ and the algorithm structurally avoids application of parallel computing methods²⁹.

Spatial subdivision methods on the GPU appear to be the most popular approaches to contact determination for particle systems. These methods partition the viewing volume into smaller, usually cuboid, sub-spaces called *cells*. The magic of any method is in the manner in which they reconcile the particle location with the cell location(s). This often requires three arrays - one array for the particle location, one for the *cell array*, and middle relation such as a *hash* or some type of *relational tree* to store the cell, or cells, each particle occupies. RCCD circumvents this middle table and fills the cell array, in parallel, directly.

The mechanics of these approaches generally fall into three categories - *atomics*, *sorts*, or *atomics/sorts*. Atomics allow

thread level synchronization but reduce the degree of parallelism¹. Parallel sorts of the relational data structure are optimal on recent GPUs because there are now built-in sorting algorithms³⁰, but they are separate data-wide processes which reduce performance. Most papers reviewed combined atomics with sorts^{2,20,31,32}.

The process of collision detection is often viewed as a two phase process of *broad* and *narrow*. In the broad phase objects that occupy the same cell, but not necessarily in contact, are collected into a *potentially colliding set*. In the narrow phase these potentially colliding objects are tested for actual contact³³. The strategy is for the broad phase to accelerate the detection process, cull collision candidates³⁴, or disserve sequential parts of the algorithm from parallel parts³⁵, thereby accelerating the narrow phase.

We are reporting on a method that performs broad detection in image space, narrow detection in the compute GPU pipeline, and is executed exclusively on the GPU. The *graphics and compute pipelines* are sequential and require synchronization. The broad phase is conducted in conjunction with *fragment rendering* so that visualization and PCS construction are concurrent³⁶. The bounding volume is built by assembling cuboids of unit side length into a larger cube. The particle location is rounded, and converted to an index, quickly locating the occupied cell. The cell occupancy of the eight corners of a particle bounding box are located in parallel by this rounded-index method and in the narrow phase is embarrassingly parallel.

3 Design and Implementation

The application was programmed with C++ in conjunction with the Vulkan API. The pipeline details are covered in the supplemental information.

The method utilizes a uniform 3-d grid centered on whole number nodes of unsigned integers. This grid is an assembly of cells that are rounded and then indexed which provide the structure for particle collision detection by spatial subdivision. Each cell is a volume element of unit side length in all three dimensions and centered on natural numbered (unsigned integer) positions with address $[X, Y, Z]$. These cells are assembled to form the larger cuboid *cell array*.

Particle information is contained in the *particle array*. Particle positions $[x, y, z]$ are expressed in floating point, positive counting numbers (real excluding zero), and represent the actual position of the particle. The convention here is that whole numbers are expressed as uppercase while floats are lowercase.

The cell at any $[X_i, Y_i, Z_i]$ extends halfway between the grid point in every direction (1),(2),(3).

$$X_i - 0.5 < X_i < X_i + 0.5 \quad (1)$$

$$Y_i - 0.5 < Y_i < Y_i + 0.5 \quad (2)$$

$$Z_i - 0.5 < Z_i < Z_i + 0.5 \quad (3)$$

Rounding the real location of a particle in $[x_i, y_i, z_i]$ results in a whole number cell location at $[X_i, Y_i, Z_i]$. For example, a particle center located at $[12.3, 14.5, 5.6]$ will be located in the cell centered at $[12, 14, 6]$ after rounding (4). Particles are not located by their centers since they may span multiple cells, or be completely enclosed by one. Detecting the location of of each region of a sphere is expensive since every point on the surface of the sphere must be tested for a containing cell. To avoid this expense, the particle is represented by a *bounding cube* called a *axis aligned bounding box* (AABB) with edge widths equal to the diameter, D or $2R$, of the particle.

$$[\text{round}(12.3), \text{round}(14.5), \text{round}(5.6)] = [12, 14, 6] \quad (4)$$

Now occupancy is determined by calculating the location of the eight corner vertices, $0 \leq c \leq 7$, of the bounding cube as illustrated by the red points in fig. 2. These 8 points naturally coincide with the maximum number of cells that can form a junction. The occupancy of each corner is determined by calculating the vectors associated with each vertex and rounding (5).

$$\vec{c}_j = \text{round}([X_j \pm P_j.R, Y_j \pm P_j.R, Z_j \pm P_j.R]) \quad (5)$$

$$0 \leq j \leq 7$$

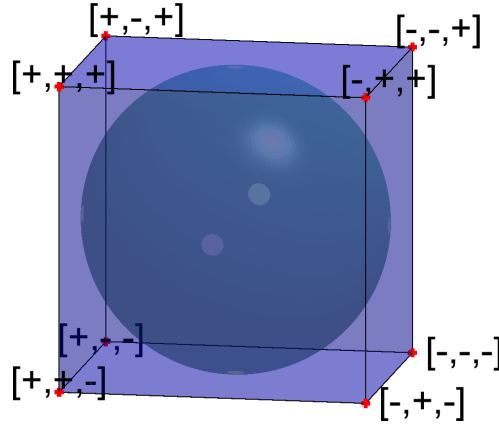


Figure 2. A particle bounded by an Axis Aligned Bounding Box (AABB) showing the eight corners.

3.1 Location Indexing

It would also be inefficient to address every cell by the 3 components of the rounded location so cell locations are converted to and from an index. Eq. (6) transforms the the rounded X, Y, Z components to an index J based on the height H and width W of the cell array. The cell array cannot contain a cell centered on zero as this would introduce negative locations, therefore the index J starts at one. The 1 based nature of the indexing account for the 1 being added to height, width, and length. Eqs. (7-9) transform the index, J , back into the 3 rounded value components. The cell array is now an array of rows equal to the maximum cell location.

It should be noted that the array-to-index transform would not be possible without rounded cells. That is, only whole number 3-d arrays result in whole number indexes. It is this property that allows such rapid location and occupancy determination. Also, cell locations must be expressed in positive numbers where no corner can exceed $WIDTH+0.5$ and be less than 0.5 in any dimension or indexing will fail.

$$J = X + (W + 1)[Y + Z(H + 1)] \quad (6)$$

$$X = \frac{J}{(W + 1)(H + 1)} \quad (7)$$

$$Y = \frac{J - [X(W + 1)(H + 1)]}{(W + 1)} \quad (8)$$

$$Z = J - (W + 1)[Y + X(W + 1)] \quad (9)$$

Fig. 3 depicts a 2-dimensional representation of particle and cell relationships. Particle 1 occupies cell $\langle 1, 1 \rangle$ at index 5 and cell $\langle 2, 1 \rangle$ at index 9 while particle 2 occupies both cells $\langle 2, 2 \rangle$ at index 10 and cell $\langle 1, 2 \rangle$ at index 6. Particle 1 will have 2 duplicates which will need to be removed to avoid duplicating collision testing.

A number of relationships between cells and particles evolve from this scheme. The total *quantity* (I_q), of the system is the number of particles contained within it. The number of cells which constitute the cell array is the *volume* (I_c). The *particle-cell density*, (max_p), is the maximum number of particles that can occupy a cell without overlap which is related to the particle radius, R . The *total collision fraction*, (F), is the total quantity of particles in collision. The *cell collision fraction*, (f), is the number of collisions occurring in a single cell. In this research the cell array is always square so that *side length*, L_s , is equal in all dimensions.

3.2 Memory Configuration

The application allocates three memory blocks called *shader storage buffer objects* (SSBO's) - the *particle array*, which stores the particle list, the *cell array* which stores the cell occupancy of particles, and the *atomic counter array* which stores a slot index into the cell array. In this application these buffers are accessible to and shared by, the compute, vertex, and fragment kernels.

Broad-phase collision detection builds the *potentially colliding set* (PCS), in the *cell array*. The cell array is then accessed by the compute block to determine narrow phase contact.

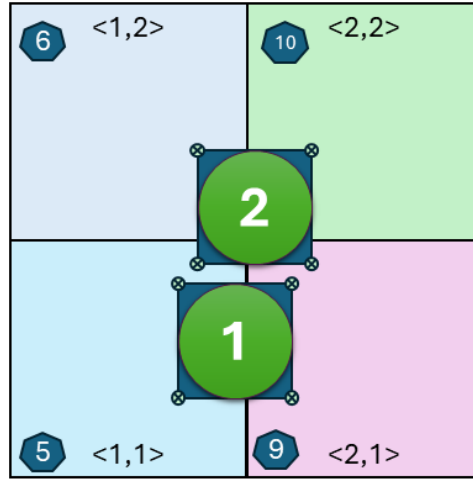


Figure 3. 2-d rounded cell illustrating cell and particle locations. Particle 1 spans cell $\langle 1,1 \rangle$ and $\langle 2,1 \rangle$ while particle 2 is completely contained in cell $\langle 2,2 \rangle$

3.3 Particle Array

The particle array is a combined *vertex buffer* and array of structures. Normally, vertices are loaded into the fixed, inaccessible, *vertex buffer* by the API. But Vulkan can point this buffer to user memory which then treats it as a vertex buffer in the vertex kernel. The particles can now be treated as vertices and processed through the highly efficient fixed function *vertex assembler* block while at the same time providing direct access to the programmable function blocks.

The particle array is an array of structures containing particle data such as location, velocity, size, etc. The length of the array is I_q , and the individual number of the particle is called the *particle ID*. The member variables of interest here is the 8 element *corner location array* which stores the location index of the particle's 8 AAB B corner locations.

3.4 Cell Array

The *cell array* is an array of arrays, where the rows hold the cell location and the columns store a list of particles corners that occupy that cell. The columns of the cell array are referred to as the *cell occupancy list*. The length of this sub-array must be determined at runtime and must be long enough to hold the maximum number of particles that can occupy the cell, max_p . The GPU does not allow dynamic allocation of device global memory, so this is memory intensive, as every cell location must accommodate the maximum number of occupying particles even though there may be few, or no, particles in occupancy for a particular cell.

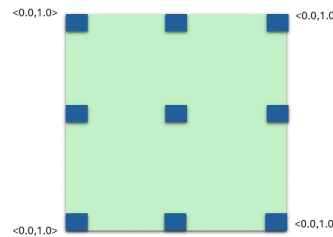


Figure 4. Point tile, representing a pixel, in normalized device space, where the purple squares are fragments.

3.5 Atomic Counter Array

The *atomic counter array* is an *coherent memory* array the same length and index addresses as the cell array. It provides a safe slot into the cell occupancy list.

The potentially colliding set (PCS) is built by adding each particle that occupies a cell to the cell's occupancy list. This is a parallel process so any number of threads can be competing to add their particle number into the same slot in the occupancy list. Each element of the atomic counter array coincides with a cell location and provides a counter for the current open position in the occupancy list. Each particle corner thread increments the value stored in the associated atomic counter array element in a thread safe manner with an `atomicAdd`.

3.6 Duplicates

There are two types of duplicates that can occur with this method - *corner duplicates* and *cell duplicates*.

In the case of *corner duplicates*, one or more corners of the bounding cube can occupy the same cell. This causes multiple corners to be added to the occupancy list of a single cell. Any particle traversing the list will see this particle multiple times and will in turn calculate a collision multiple times. In fig. 3, notice that particle 1 has duplicate corners in cell 5 and 9.

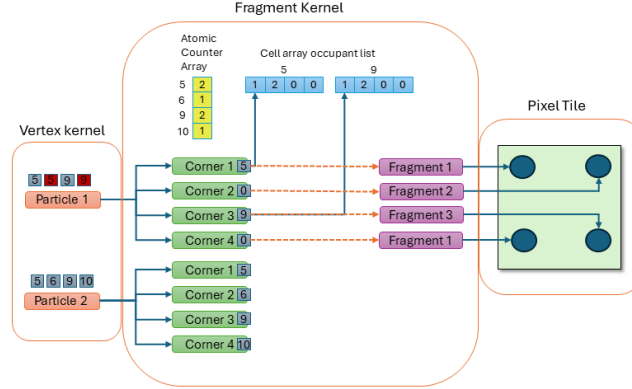


Figure 5. Broad phase PCS construction.

Cell duplicates occur when two particles span the same cells, which causes the same two particles to be paired in multiple cell locations. This is illustrated in fig. 3 where both particle 1 and particle 2 span cells 5 and 9.

3.7 Fragment Processing

Vulkan can process *graphics primitives* as points, lines, or triangles - among others. In this application they are processed as point primitives where the topology is set to VK_PRIMITIVE_TOPOLOGY_POINT_LIST.

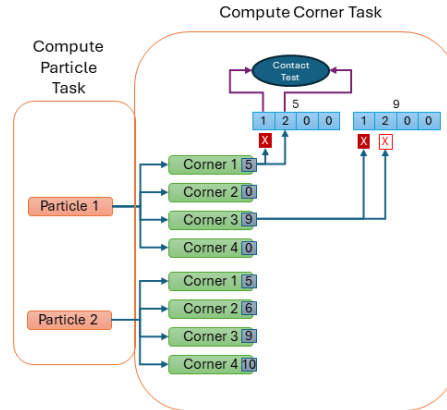


Figure 6. Narrow phase process - contact determination.

The size of a pixel, as a square not a circle, is configured by setting the `gl_PointSize` vertex kernel *built-in* to the desired size. The fragment kernel does not actually render a pixel but writes fragments of a pixel into a *point-tile* of unit side length as illustrated in fig. 4. When the point size is set to 3.0 the pattern illustrated in fig. 4 is written by 9 fragment *invocations*. This tile is fed to a smoothing fixed function kernel at the end of the graphics pipeline to be rendered as an actual pixel(s). The smoothing fixed function block determines the depth and size of the pixel and if it will be rendered or culled.

Each fragment drawn to the tile has a unique location and these are used to identify each individual fragment invocation.

3.8 Process

The particle-cell relationship follows fig. 3 which is in two dimensions, instead of three, reducing the number of corners to four instead of eight.

The graphics pipeline *broad phase* process is illustrated in fig. 5. A vertex kernel invocation is launched for each particle in parallel. Each invocation calculates the corner locations of the particle and writes them to the particle array corner location

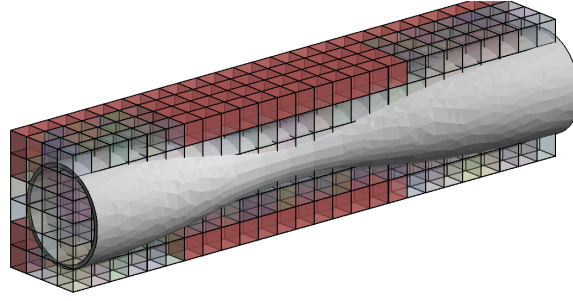


Figure 7. A converging-diverging nozzle showing cells that can be "out indexed" in red.

array. If there are corner duplicates they are culled as illustrated by the red squares above the orange particle blocks. For example, in fig. 3, particle 1 has two duplicate corners in 5 and 9, while particle 2 has no duplicate corners.

Table 1. Particle radius versus particles per cell max_p .

Radius R Range	max_p
0.16-0.2	8
0.11-0.15	27
0.91-0.11	64
0.071-0.09	125
0.061-0.07	216
0.051-0.06	343
0.041-0.05	729
0.031-0.04	1331
0.021-0.03	3375
0.011-0.02	12167

The vertex kernel then spawns 8 fragment invocations - one for each corner location for the current particle as illustrated by the green blocks. Corner locations with a zero in the address are not processed for location assignment. Each fragment task determines the index location for its corner, as discussed in sec. 3.7. An atomic add returns the current open slot which increments the atomic counter array at the same location. The yellow blocks representing the atomic counter array show that 2 corners have been written to location 6 and 9 from particle 1 and one corner to each of 6 and 10 from particle 2. Likewise, each particle ID is stored in the cell array cell occupancy list at locations 6 and 9.

The fragment kernel draws 4 fragments of a pixel concurrent with the corner thread locating as represented by the orange dotted lines.

The compute pipeline *narrow phase* is illustrated in fig. 6. Once the vertex and fragment stages are complete every cell contains a list of the particle corners contained there. The cell array is then passed to the compute phase which launches a task for each particle shown as the orange blocks in the figure. The particle task then launches 8 consecutively numbered tasks each representing a corner location in the particle array corner location array. The compute corner thread reads the particle array corner location array and uses that as a pointer into the cell array cell occupancy list.

The corner thread then loops over each element of the cell array cell occupancy list reading the particle ID of the those that occupy that cell. The logic ignores its own number as shown by the red block with a white X. It also passes over cell duplicates which are shown as a red X in a red box. In fig. 3, particle 1 is collocated with particle 2 indicating that they are potentially colliding. Contact determinations is performed for potentially colliding particles utilizing a simple distance test. If the collision is true, the thread can then calculate the collision response, otherwise it continues to loop over the cell array cell occupancy list.

3.9 Memory

This method is memory intensive as the number of cells increase cubically ($O(3)$ complexity). But in the current market speed is more expensive than memory, and memory continues to fall in price while speed is at a relative economic standstill. That being said there are a number of approaches that can optimize memory use.

One solution that can conserve a substantial quantity of memory is *memory indexing*. A separate single array called an *index hash table* can be added to the process. This table spans all locations as the cell array did previously but now it only contains locations for cells that are in range or valid. Invalid cells are depicted as the red cubes illustrated in fig. 7. Another index hash table contains all locations but points only to valid cells.

Table 2. Compute (*mcpt*) and graphics(*mgpt*), processing times. With maximum frames per second (*maxfps*), minimum seconds per frame (*minspf*), application render rate, performance by number of particles where particle-cell density, *max_p* is 8, collision fraction, *F*, is 0.5, and *mmrr* is 7.18×10^{-2} ms.

Particles in Dataset	Compute (Narrow) <i>mcpt</i> (s)	Graphics (Broad) <i>mgpt</i> (s)	<i>maxfps</i> (s ⁻¹)	<i>minspf</i> (s)	<i>arr</i>
32	6.14e-06	1.02e-05	6512	1.54e-04	46.7816
82944	3.70e-04	3.02e-04	1179	8.48e-04	8.4698
115712	5.15e-04	4.15e-04	891	1.12e-03	6.4009
246784	1.26e-03	8.81e-04	420	2.38e-03	3.0172
508928	2.71e-03	1.80e-03	206	4.85e-03	1.4799
754688	4.06e-03	2.68e-03	140	7.14e-03	1.0057
1000448	5.41e-03	3.54e-03	106	9.43e-03	0.7615
1246208	7.06e-03	4.44e-03	83	1.20e-02	0.5963
1360896	7.71e-03	4.85e-03	77	1.30e-02	0.5532
1508352	8.56e-03	5.36e-03	69	1.45e-02	0.4957
1623040	8.81e-03	5.72e-03	66	1.52e-02	0.4741
1754112	9.98e-03	6.25e-03	60	1.67e-02	0.4310
1901568	1.03e-02	6.70e-03	57	1.75e-02	0.4095
2016256	1.10e-02	7.10e-03	54	1.85e-02	0.3879
2163712	1.23e-02	7.65e-03	49	2.04e-02	0.3520
2343936	1.30e-02	8.26e-03	46	2.17e-02	0.3305
2540544	1.40e-02	8.91e-03	43	2.33e-02	0.3089
2638848	1.51e-02	9.29e-03	40	2.50e-02	0.2874
2753536	1.57e-02	9.74e-03	39	2.56e-02	0.2802
2900992	1.58e-02	1.02e-02	38	2.63e-02	0.2730
3015680	1.74e-02	1.07e-02	35	2.86e-02	0.2514
3130368	1.73e-02	1.10e-02	35	2.86e-02	0.2514
3245056	1.79e-02	1.14e-02	34	2.94e-02	0.2443
3359744	1.85e-02	1.18e-02	33	3.03e-02	0.2371
3507200	1.93e-02	1.24e-02	31	3.23e-02	0.2227
3621888	1.99e-02	1.27e-02	30	3.33e-02	0.2155
3752960	2.16e-02	1.33e-02	29	3.45e-02	0.2083
3867648	2.11e-02	1.36e-02	29	3.45e-02	0.2083
4015104	2.19e-02	1.42e-02	27	3.70e-02	0.1940
4129792	2.27e-02	1.45e-02	27	3.70e-02	0.1940
4260864	2.35e-02	1.50e-02	26	3.85e-02	0.1868
4375552	2.41e-02	1.54e-02	25	4.00e-02	0.1796

This application will accommodate particles of different sizes. The uniform grid represented by the cell array has unit width so that only particles with a diameter of less than one can occupy a cell. If the largest particle is set to a diameter of 1 then the relative diameter of a smaller particle is that diameter divided by the largest particle diameter.

The size of a particle also impacts the number of particles that can be contained in a cell. The particle quantity can be increased in two ways - increase the number of particles per cell, *I_q*, or increase the number of cells, *L_s*, or both. It will be shown subsequently that the latter is higher performing.

Table 1 shows the relationship between the maximum number of particle that can fit in a cell versus the maximum range of particle radii for that population. This table assumes that the particles fit inside the cell without overlap and without intersecting other particles by providing a margin that is 15% of the radius.

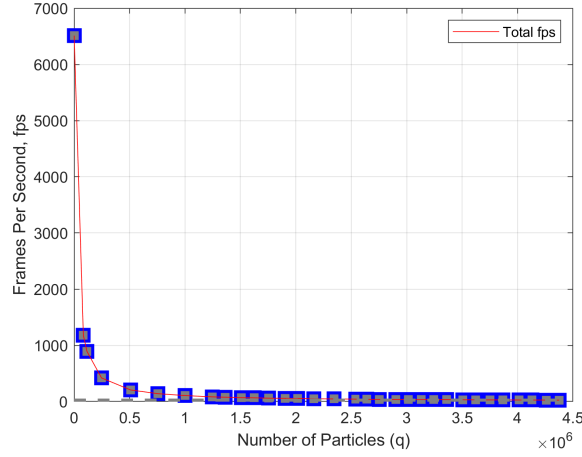


Figure 8. Frame per (*spf*) for 0.5 collision density with 8 particles per cell verses number of particles where particle-cell density, $\max_p$ is 8, collision fraction, F , is 0.5, and mmrr is 4.35×10^{-3} ms

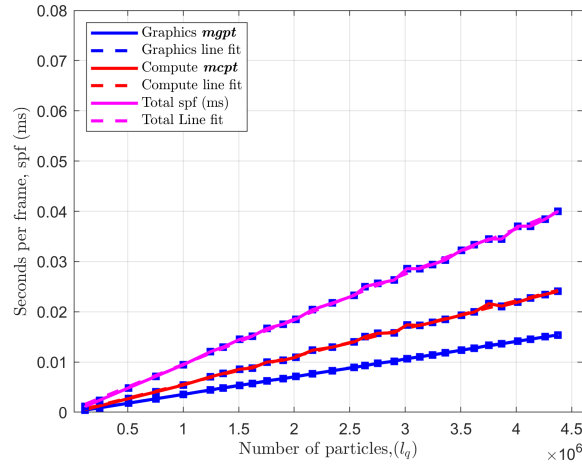


Figure 9. Particle Quantity Benchmark (PQB). Seconds per frame (ms) versus number of particles. The blue lines represent the performance of the graphics pipeline (broad phase), *mgpt*. The orange line represents the performance of the compute pipeline (narrow), *mcpt*. The purple lines mark the total performance, *mtpt*. The dashed lines are linear fit lines. The fit equations are, graphics: $3.51 \times 10^{-9}q + 2.97 \times 10^{-5}$, compute: $5.53 \times 10^{-9}q + 1.79 \times 10^{-5}$, total: $8.99 \times 10^{-9}q + 5.38 \times 10^{-4}$. Conditions are particle-cell density, $\max_p$ is 8, collision fraction, F , is 0.5, and mmrr is 7.18×10^{-5} ms

4 Methodology

Development and testing are performed on the following platforms.

- Lenovo® ThinkPad™ P16 Gen 2 (16) Mobile Workstation, 13th Generation Intel® Core™ i9-13980HX processor with a NVIDIA® RTX™ 3500 Ada Generation Laptop GPU 12GB GDDR6.

This is an exact collision detection method and it therefore requires verification and performance testing. Each step of this process is documented here with publicly available code and data. The overall process entails :

1. generating data,
2. verifying that data,
3. verifying the application against that data,
4. performance testing against the data,
5. generating plots and tables,

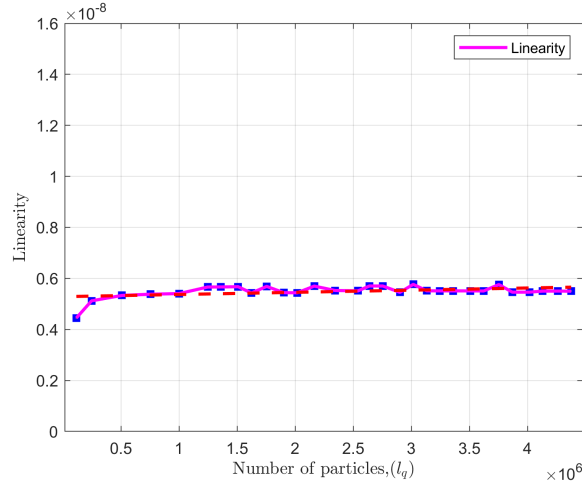


Figure 10. Linearity of particle quantity benchmark (PQB) compute pipeline where linearity is $\frac{mcpt}{q}$. Linearity versus number of particles for compute pipeline, **mcpt**. The orange dashed line is the linear fit line. The fit equation is, compute: Linear fit: $8.50e-17q + 5.28e-09$ Conditions are particle-cell density, **max_p** is 8, collision fraction, **F**, is 0.5, and **mmrr** is 7.18×10^{-5} ms

6. vetting data and code.

The *particle quantity benchmark* (PQB) tests the processing time for an increasing number of particles while collision and particle-cell density are held constant. The *collision fraction benchmark* (CFB) tests a varying number of collisions while holding the number of particles and the particle-cell density constant. The *particle cell density benchmark* (PCD) measures the execution times for a decreasing number of cells while increasing the particle destiny in those cells while maintaining particle quantity.

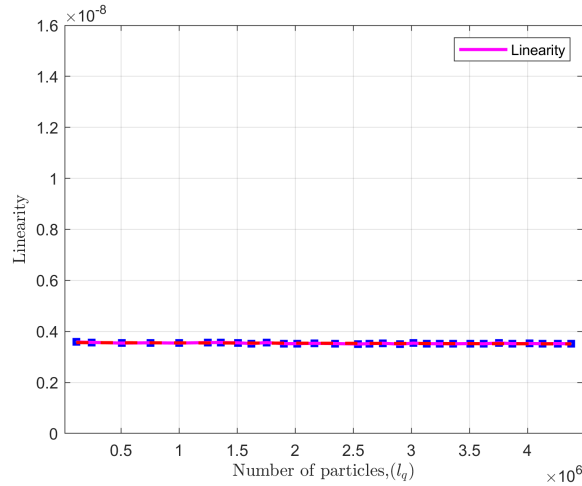


Figure 11. Linearity of particle quantity benchmark graphics (PQB) where linearity is $\frac{mgpt}{q}$. Linearity versus number of particles for graphics pipeline, **mgpt**. The orange dashed line is the linear fit line. The fit equation is, graphics: $-1.1 \times 10^{-17}q + 3.56 \times 10^{-9}$ Conditions are particle-cell density, **max_p** is 8, collision fraction, **F**, is 0.5, and **mmrr** is 7.18×10^{-5} ms

Particle Quantity Benchmark (PQB)

1. Number of particles:variable
2. Particle cell density:fixed
3. Number of collisions:fixed

Collision Fraction Benchmark (CFB)

1. Number of particles:variable
2. Particle cell density:variable
3. Number of collisions:fixed

Particle Cell Density Benchmark (PCD)

1. Number of particles:fixed
2. Particle cell density:fixed
3. Number of collisions:variable

4.1 Benchmarking

The application provides a simple set of non-dimensional metrics for performance comparison across hardware platforms and with other particle collision detection methods. The test suite measures the processing time for the CPU labeled *mcput*, the graphics pipeline time *mgpt*, and the compute pipeline *mcpt* with the total GPU processing time, *mtpt*, being the sum of graphics and compute (*mgpt+mcpt*).

A light-weight data set generator produces multiple datasets for each test type. The particle collision detection application written for this paper automatically runs over each of these on the target hardware-software combination for 60 seconds. For each dataset the applications records the minimum render time, *minspf*, of the 60 timings, where *minspf* is the inverse of the maximum frame rate, *maxfps*.

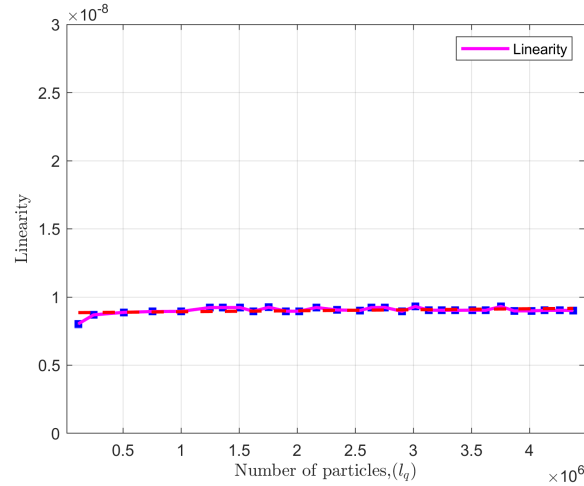


Figure 12. Linearity of particle quantity benchmark (PQB) for graphics and compute pipelines, where linearity is $\frac{mtpt}{q}$. Linearity versus number of particles total, *mtpt*. The orange dashed line is the linear fit line. The fit equation is, total: $7.32 \times 10^{-17}q + 8.84 \times 10^{-9}$ Conditions are particle-cell density, *max_p* is 8, collision fraction, *F*, is 0.5, and *mmrr* is 7.18×10^{-5} ms

Another application records the time it takes the CPU-GPU hardware to render a single triangle where vertices are embedded in the kernel. This is the benchmark referred to as *minimum machine render rate* (*mmrr*).

Finally, the application rate ratio (*arr*) is calculated as percentage derived by dividing the *mmrr* by the *minspf*.

5 Results

The current version of the software is not fully parallel owing to the fact that neither the compute or graphics kernel launch corner threads. A fully parallel version awaits Vulkan extension allowing shared memory to process both cell and corner duplicates.

Table 2 lists the best case total performance which includes the number of particles processed in each dataset, the minimum compute, graphics times, *mcpt*, *mgpt*, the maximum frame rate, *maxfps*, minimum seconds per frame, *minspf*, and the application render ratio, *arr*. Table 2 reports that this version of RCCD performs realtime at 25 fps for 4,375,552 particles. The CPU time *mcpt*, appears to be almost twice the GPU time, *mgpt*. It may be that cell duplicate processing in the compute kernel is more extensive than that of the graphics pipeline.

5.1 Particle Quantity Benchmark (PQB)

Results are plotted in frames per second in fig. 8 and fig. 9 by seconds per frame. Fig. 8 shows a rapid fall off to about 1 million particles and thereafter appears to be linear.

Fig. 9 shows graphics, compute and total processing time. The orange line represents the compute time *mcpt*, the blue line is the graphics time *mgpt*, and the purple line reports the total, *mtpt*. The dashed lines of the same colors are the linear fit lines.

Both fig. 8 and fig. 9 indicate that the graphics time is less than compute and that both are clearly linear. For graphics, compute, and total times the number of particles increases at a greater rate that does the time.

We report on the linearity of the methods shown in figs. 11, 10 and 12. Total linearity is formed from the superposition of the inferred graphics and compute linearity. These plots show an unmistakable linear relationship between the number of particles and the time to process them.

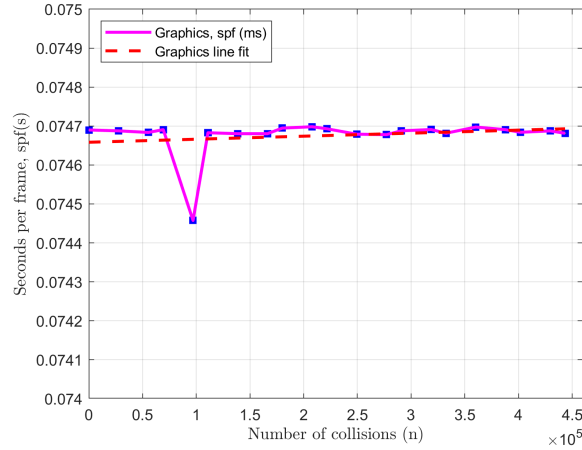


Figure 13. Collision Fraction Benchmark (CFB), graphics execution time versus number of collisions. For 443392 particles. The collision fraction, F varies from 0.0 to 1.0 in 0.05 increments and where particle-cell density, $\max_p$ is 8.

Both graphics and compute have fractional slopes but the graphics *mgpt* is very nearly constant. The total in fig. 12, *mtpt* also exhibits a fractional slope.

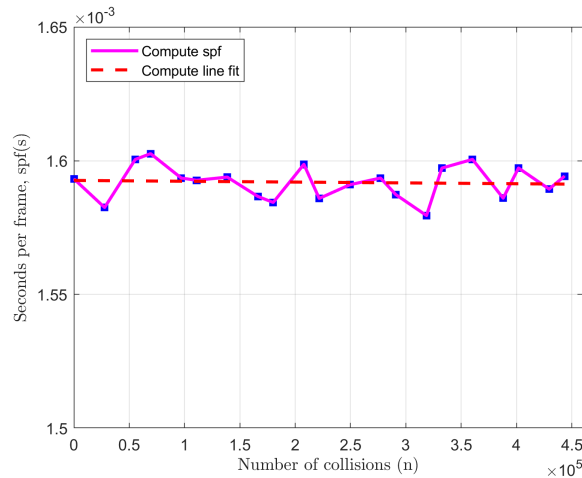


Figure 14. Collision Fraction Benchmark (CFB), compute execution time versus number of collisions. For 443392 particles. The collision fraction, F varies from 0.0 to 1.0 in 0.05 increments and where particle-cell density, $\max_p$ is 8.

5.2 Collision Fraction Benchmark (CFB)

The collision fraction benchmark reports on increasing the collision fraction, f , while holding the quantity of particles constant. Fig. 13 and fig. 14 are plots of compute and graphics execution times for various f .

In the Collision Fraction Benchmark test a constant 443,392 particles were tested over a collision fraction that varied from 0.0 to 1.0 in 0.05 increments where the max_p was 8 per cell. Both fig. 13 and fig. 14 plots imply that RCCD is very insensitive to the number of collisions.

Table 3. Particle radius versus particles per cell max_p .

L_s	Radius R Range	max_p	$mcpt(s)$	$mgpt(s)$
91	0.16-0.2	8	0.03	0.02
61	0.11-0.15	27	0.07	0.02
46	0.91-0.11	64	0.15	0.02
37	0.071-0.09	125	0.27	0.02
31	0.061-0.07	216	0.47	0.02
27	0.051-0.06	343	0.71	0.02
21	0.041-0.05	729	1.44	0.02
18	0.031-0.04	1331	2.56	0.02
13	0.021-0.03	3375	6.26	0.02
9	0.011-0.02	12167	22.57	0.02

5.3 Particle Cell Density Benchmark (PCB)

The particle cell density benchmarks fig. 15 and fig. 16 plot the reaction of the compute and graphics pipeline to increasing particles per cell, max_p . Table 3 supports this plots by itemizing side length, radius ranges and particles per cell.

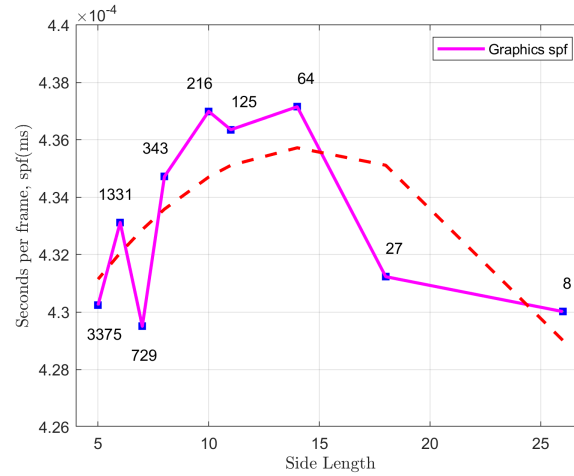


Figure 15. Particle Cell Density Benchmark graphics(PCB). Measures the execution time as the number of particles per cell increases. The numbered points are the particles per cell, max_p . Execution time in seconds per frame versus the side length, L_s . The side length squared equals the number of cells, l_c . Table 3 lists the data points showing the number of particles per cell, max_p , as the side length decreases. Graphics time, $mgpt$, to process 117648 particles, as particle per cell, max_p , increase from 8 to 3375.

According to fig. 15 the graphics pipeline seems to have a peak at a more equal distribution of particle and cell quantities. This is somewhat unexpected as nothing changes in the broad logic. A memory block of 91 rows, and 8 columns, is the same as a single row of 91x8=728 elements, so storing corner locations will be the same. And, the graphics kernel only processes 8 corners per particle regardless of max_p .

But the same is not true for the compute as shown in fig. 16. The number of particle threads launched remains the same, but each thread must search for more and more particles per cell as max_p increases. In the worst case, 117,648 threads were forced to search only 8 locations with 14,706 particles per cell ($\frac{11648}{8}$) which took more than 25 seconds. One might have predicted that there would be a balance between number of cells and particles per cell, but the plot clearly indicates that fewer particles per cell provides higher performance.

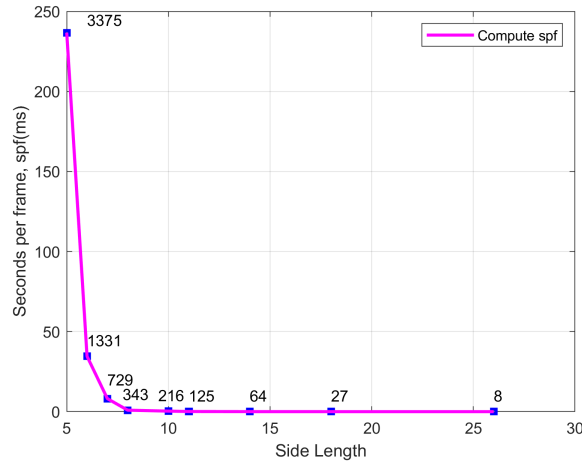


Figure 16. Particle Cell Density Benchmark compute (PCB). Measures the execution time as the number of particles per cell increases. The numbered points are the particles per cell, max_p . Execution time in seconds per frame versus the side length, L_s . The side length squared equals the number of cells, l_c . Table 3 lists the data points showing the number of particles per cell, max_p , as the side length decreases. Compute time, $mgpt$, to process 117648 particles, as particle per cell, max_p , increase from 8 to 3375.

6 Summary and Future Work

This system is very simple and the code works for any particle system requiring exact detection. The speed of this single CPU-GPU application is transformative. It changes the landscape of particle research by increasing availability. The method is also nascent and further work, such as introducing new Vulcan extensions promises to make it ever faster. We envision single CPU's attached to multiple GPU's that act as a sort of mini-HPC providing researchers an inexpensive middle-ware between single CPU systems and HPC.

7 Data Availability

The authors declare that the data and code supporting the findings of this study are available at:
<https://github.com/fieldparticle/FPIBGP>.

References

1. Green, S. [Particle Simulation Using CUDA](#) (2010).
2. Goswami, P., Schlegel, P., Solenthaler, B. & Pajarola, R. [Interactive SPH Simulation and Rendering on the GPU](#). In *Eurographics/ ACM SIGGRAPH Symposium on Computer Animation (2010)* (2010).
3. Govindaraju, N. [Efficient Visibility-Based Algorithms for Interactive Walkthrough Shadow Generation and Collision Detection](#). Ph.D. thesis, Chapel Hill (2005).
4. Oat, C., Barczak, J. & Shopf, J. [Efficient Spatial Binning on the GPU](#). *AMD Tech. Rep.* (2009).
5. Hyesoon, K., Vuduc, R., Bahsorkhi, S. & Wen-mei Hwu, J. C. [Performance Analysis and Tuning for General Purpose Graphics Processing Units \(GPGPU\)](#). No. 20 in *Synthesis Lectures on Computer Architecture* (Synthesis, 2012).
6. Owens, J. D. *et al.* [A Survey of General-Purpose Computation on Graphics Hardware](#). *Comput. Graph. Forum* **26**, 80–113, DOI: [10.1111/j.1467-8659.2007.01012.x](#) (2007).
7. [Advancing GPU-Driven Rendering with Work Graphs in Direct3D 12](#) (2024).
8. Zhu, H., Zhou, Z., Yang, R. & Yu, A. [Discrete Particle Simulation of Particulate Systems: Theoretical Developments](#). *Chem. Eng. Sci.* **62**, 3378–3396, DOI: [10.1016/j.ces.2006.12.089](#) (2007).
9. Liu, M. B. & Liu, G. R. [Smoothed Particle Hydrodynamics \(SPH\): An Overview and Recent Developments](#). *Arch. Comput. Methods Eng.* **17**, 25–76, DOI: [10.1007/s11831-010-9040-7](#) (2010).
10. He, Y. *et al.* [A GPU-based Coupled SPH-DEM Method for Particle-Fluid Flow with Free Surfaces](#). *Powder Technol.* **338**, 548–562, DOI: [10.1016/j.powtec.2018.07.043](#) (2018).
11. Mittal, A., Mangadoddy, N. & Banerjee, R. [Development of Three-Dimensional GPU DEM Code–Benchmarking Validation and Application in Mineral Processing](#). *Comput. Part. Mech.* **10**, 1533–1556, DOI: [10.1007/s40571-023-00571-4](#) (2023).
12. Bell, J. [A SURVEY OF GPU-ACCELERATED PARTICLE COLLISION DETECTION](#) (2024).
13. Hoff, K. & Zaferakis, A. [Fast 3D Geometric Proximity Queries between Rigid and Deformable Models Using Graphics Hardware Acceleration](#). Tech. Rep. Technical Report TR02-004,, UNC Chapel Hill, Department of Computer Science (2017).
14. Latta, L. [Building a Million Particle System](#) (2004).
15. Rubio-Largo, S. M., Lind, P. G., Maza, D. & Hidalgo, R. C. [Granular Gas of Ellipsoids: Analytical Collision Detection Implemented on GPUs](#). *Comput. Part. Mech.* **2**, 127–138, DOI: [10.1007/s40571-015-0042-y](#) (2015).
16. Gan, J., Zhou, Z. & Yu, A. [A GPU-based DEM Approach for Modelling of Particulate Systems](#). *Powder Technol.* **301**, 1172–1182, DOI: [10.1016/j.powtec.2016.07.072](#) (2016).
17. Xu, R., Kang, L. & Tian, H. [A G-Octree Based Fast Collision Detection for Large-Scale Particle Systems](#). In *2012 International Conference on Computer Science and Electronics Engineering*, 269–273, DOI: [10.1109/ICCSEE.2012.98](#) (IEEE, Hangzhou, Zhejiang, China, 2012).
18. Lo, S.-H., Lee, C.-R., Chung, Y.-C. & Chung, I.-H. [A Parallel Rectangle Intersection Algorithm on GPU+CPU](#). In *2011 11th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing*, 43–52, DOI: [10.1109/CCGrid.2011.13](#) (IEEE, Newport Beach, CA, USA, 2011).
19. Knoll, A., Morley, R. K., Wald, I., Leaf, N. & Messmer, P. [Efficient Particle Volume Splatting in a Ray Tracer](#). In Haines, E. & Akenine-Möller, T. (eds.) *Ray Tracing Gems*, 533–541, DOI: [10.1007/978-1-4842-4427-2_29](#) (Apress, Berkeley, CA, 2019).
20. Zheng, J., An, X. & Huang, M. [GPU-based Parallel Algorithm for Particle Contact Detection and Its Application in Self-Compacting Concrete Flow Simulations](#). *Comput. & Struct.* **112–113**, 193–204, DOI: [10.1016/j.compstruc.2012.08.003](#) (2012).
21. Morvan, T., Reimers, M. & Samset, E. [Efficient Image-Based Proximity Queries with Object-Space Precision](#). *Comput. Graph. Forum* **31**, 62–74, DOI: [10.1111/j.1467-8659.2011.02084.x](#) (2012).
22. Jang, H. & Han, J. [Fast Collision Detection Using the A-buffer](#). *The Vis. Comput.* **24**, 659–667, DOI: [10.1007/s00371-008-0246-8](#) (2008).
23. Hemanth, D. J., Elhosney, M. & Nguyen, T. N. [Advances in Parallel Computing Technologies and Applications](#) (IOS Press, Incorporated, Amsterdam, NETHERLANDS, THE, 2021).
24. Hockney, R. W. [Computer Simulation Using Particles](#) (Taylor & Francis Group, LLC, 1988).

25. Bader, M. *et al.* (eds.) [Parallel Computing: Accelerating Computational Science and Engineering \(CSE\)](#). No. v. 25 in *Advances in Parallel Computing* (IOS Press, Amsterdam, 2014).
26. Yuen, D. A. *et al.* (eds.) [GPU Solutions to Multi-scale Problems in Science and Engineering](#). *Lecture Notes in Earth System Sciences* (Springer Berlin Heidelberg, Berlin, Heidelberg, 2013).
27. Nvidia. [Rigid Body Collision — NVIDIA PhysX SDK 4.1 Documentation](#).
28. Engelke, W., Lawonn, K., Preim, B. & Hotz, I. [Autonomous Particles for Interactive Flow Visualization](#). *Comput. Graph. Forum* **38**, 248–259, DOI: [10.1111/cgf.13528](#) (2019).
29. Sigurgeirsson, H., Stuart, A. & Wan, W.-L. [Algorithms for Particle-Field Simulations with Collisions](#). *J. Comput. Phys.* **172**, 766–807, DOI: [10.1006/jcph.2001.6858](#) (2001).
30. Luitjens, J. [Faster Parallel Reductions on Kepler](#). *NVidia Dev. Simulation/Modeling/Design* (2014).
31. Franklin, W. R. & Magalhães, S. V. G. [Parallel Intersection Detection in Massive Sets of Cubes](#). In *Proceedings of the 6th ACM SIGSPATIAL Workshop on Analytics for Big Geospatial Data*, 20–26, DOI: [10.1145/3150919.3150921](#) (ACM, Redondo Beach CA USA, 2017).
32. Lo, S.-H., Lee, C.-R., Chung, I.-H. & Chung, Y.-C. [Optimizing Pairwise Box Intersection Checking on GPUs for Large-Scale Simulations](#). *ACM Transactions on Model. Comput. Simul.* **23**, 1–22, DOI: [10.1145/2499913.2499918](#) (2013).
33. Ericson, C. [Real-Time Collision Detection](#) (CRC Press, 2004), 1 edn.
34. Kroiss, R. R. [Collision Detection Using Hierarchical Grid Spatial Partitioning on the GPU](#). Ph.D. thesis, University of Wisconsin, Madison (2009).
35. Nyland, L., Harris, M. & Prins, J. [Chapter 31. Fast N-Body Simulation with CUDA](#). *GPU GEMS 3* (2003).
36. Kipfer, P., Segal, M. & Westermann, R. [UberFlow: A GPU-Based Particle Engine](#). *Graph. Hardw.* DOI: [10.1145/1186223.1186253](#) (2004).